

Duale Hochschule Baden-Württemberg Stuttgart

Modul: Cloud Computing II

Travel Assistant

Eine cloud-native KI-Reiseplanungsanwendung mit
automatisiertem Deployment, Monitoring und Disaster Recovery

Schriftliche Ausarbeitung zur Projektarbeit

Verfasser: Timothy Kugler, 2348056

Modul: Cloud Computing II

Abgabedatum: 3. Mai 2026

Repository: github.com/timothykugler/travel-assistant

Inhaltsverzeichnis

1	Einleitung	3
1.1	Kontext und Motivation	3
1.2	Zielsetzung	3
1.3	Problemstellung	3
2	Technische Grundlagen	4
2.1	Cloud-Service-Modelle	4
2.2	Eingesetzte Technologien	4
2.3	Begründung der Technologieauswahl	5
3	Architektur und Umsetzung	7
3.1	Gesamtarchitektur	7
3.2	Request-Pfad und Deployment-Flüsse	8
3.3	Entwicklung des Deployment-Modells	9
3.4	Monitoring-Stack	9
4	Fokus-Feature: Disaster Recovery	9
4.1	Leitidee: zwei Recovery-Pfade	9
4.2	Compute-Recovery über die MIG	10
4.3	Datenrecovery	10
4.4	Begründung einzelner Designentscheidungen	11
4.5	Point-in-Time Recovery (Ausblick)	11
5	Ergebnisse und Diskussion	12
5.1	Umgesetzter Systemumfang	12
5.2	Architektonische Iterationen	12
5.3	Beobachtete Probleme und Lessons Learned	13
5.4	Grenzen der aktuellen Lösung	13
6	Fazit	13
	Referenzen	14

1 Einleitung

1.1 Kontext und Motivation

Die Planung einer Reise ist auch im Jahr 2026 trotz einer Vielzahl spezialisierter Online-Dienste aufwändig. Informationen über Ziele, Aktivitäten und Zeitpläne liegen verteilt vor und müssen von Reisenden manuell zusammengeführt werden. Gleichzeitig haben sich generative KI-Modelle zu einer tragfähigen Basis für konversationelle Assistenzsysteme entwickelt. Vor diesem Hintergrund entstand die Projektidee eines dialogorientierten Reiseassistenten, der Nutzerinnen und Nutzern bei der Planung ganzer Reisen hilft, Gespräche speichert und personalisierte Vorschläge liefert.

Das Modul Cloud Computing II fokussiert nicht nur die Funktionalität einer Anwendung, sondern vor allem ihre *Cloud-native* Umsetzung. Der Projektkontext umfasst die Nutzung aller drei Cloud-Ebenen (IaaS, PaaS, SaaS), eine automatisierte Infrastrukturbereitstellung mit Terraform, eine Konfigurationsautomatisierung mit Ansible sowie ein erkennbares Monitoring-Konzept. Der vorliegende Projektbericht beschreibt die Umsetzung eines entsprechenden Systems unter dem Namen *Timmy's Travel Assistant* auf der Google Cloud Platform (GCP).

1.2 Zielsetzung

Ziel des Projekts war die Entwicklung einer containerbasierten Web-Anwendung, die folgende Eigenschaften erfüllt:

- **Funktional:** ein KI-basierter Reiseassistent mit Nutzerkonten, persistenten Konversationen und speicherbaren Reiseplänen.
- **Architektonisch:** Einsatz aller drei Cloud-Ebenen (SaaS, PaaS, IaaS) mit klarer Verantwortungstrennung.
- **Betrieblich:** vollständig automatisierte Bereitstellung über Terraform, Ansible und GitHub Actions mit reproduzierbaren Konfigurationen.
- **Resilient:** ein Fokus-Feature *Disaster Recovery*, das sowohl Compute-Ausfälle über eine Managed Instance Group als auch Datenverluste in Firestore über ein Backup/Restore-Konzept behandelt.
- **Beobachtbar:** Monitoring mit Prometheus, Grafana und Loki, einschließlich eines dedizierten Integritätschecks für die Datenbank.

1.3 Problemstellung

Aus technischer Sicht stellten sich im Projekt vier Kernfragen, die die Architektur wesentlich prägten:

1. **Wie wird ein stabiler öffentlicher Endpunkt erreicht, obwohl einzelne VMs selbstheilend ausgetauscht werden dürfen?**
2. **Wie wird erreicht, dass selbst bei Absturz der Applikation die Überwachung weiterläuft und den Fehlerzustand anzeigt**
3. **Wie wird eine automatische Wiederherstellung der exakt gleichen Applikation erreicht?**
4. **Wie wird erreicht, dass Zwischenstände von Daten gesichert und wiederhergestellt werden können?**

Die Beantwortung dieser Fragen leitete sowohl die Infrastrukturgestaltung als auch die Gliederung dieser Ausarbeitung. Die folgenden Kapitel entwickeln die Lösung von den technologischen Grundlagen (Kapitel 2) über die Architektur und Umsetzung (Kapitel 3) bis hin zum Fokus-Feature *Disaster Recovery* (Kapitel 4), einer Diskussion der Ergebnisse (Kapitel 5) und einem Fazit (Kapitel 6).

2 Technische Grundlagen

2.1 Cloud-Service-Modelle

Die folgende Tabelle fasst die wesentlichen Eigenschaften der drei Cloud-Service-Modelle und das im Projekt gewählte Mapping zusammen.

Ebene	Definition	Im Projekt verwendet
SaaS	Fertige Software als Dienst, meist über APIs konsumiert.	Google Gemini API (Gemini 3.1 Flash-Lite) für die KI-Antworten.
PaaS	Verwaltete Plattformdienste ohne eigene Infrastrukturverantwortung.	Google Firestore (NoSQL), Google Artifact Registry für Docker-Images, Google Cloud Scheduler für geplante Backups, Google Cloud Storage für Backup- und Konfigurationsartefakte.
IaaS	Bereitstellung virtueller Maschinen, Netzwerk- und Speicherressourcen.	Google Compute Engine (App-MIG + Monitoring-VM), Load Balancer, Persistent Disks.

Tabelle 1: Zuordnung der Projekt-Komponenten zu den Cloud-Service-Modellen.

2.2 Eingesetzte Technologien

- **Backend:** Python 3.13 mit FastAPI (`0.115`), Pydantic v2 und Uvicorn als ASGI-Server. E-Mail/Passwort-Authentifizierung mit serverseitig ausgestellten JWTs über `python-jose`, Prometheus-Metriken über den `prometheus-fastapi-instrumentator`.
- **Frontend:** React + Vite, ausgeliefert als statische Dateien über Nginx. Das Frontend hält das JWT im Browser und sendet es als Bearer-Token an die API.
- **Datenhaltung:** Google Firestore (Native Mode) im Multi-Dokumenten-Modell (siehe Tabelle 2).

Collection	Inhalt
<code>users/{uid}</code>	Nutzerprofil (<code>display_name</code> , <code>email</code>)
<code>users/{uid}/conversations/{id}</code>	Chatverläufe mit dem Assistenten
<code>users/{uid}/trips/{id}</code>	Gespeicherte Reisepläne
<code>rate_limits/{uid}</code>	Tägliches Gemini-Request-Kontingent je Nutzer
<code>analytics/daily_usage</code> , <code>analytics/system</code>	Aggregierte Metriken und Seed-Markierung

Tabelle 2: Firestore-Datenmodell.

- **KI-Modell:** Google Gemini API mit *Gemini 3.1 Flash-Lite* für die Reiseassistentz.
- **Container und Orchestrierung:** Docker + Docker Compose, ein eigener Nginx als Reverse Proxy pro App-VM.

- **Infrastructure as Code:** Terraform (Provider `hashicorp/google ~> 5.0`) für Netzwerk, VMs, Load Balancer, Firestore, IAM und Cloud Scheduler.
- **Konfigurationsmanagement:** Ansible (Playbook + Rolle) ausschließlich für die persistente Monitoring-VM.
- **CI/CD:** GitHub Actions (Build, Push, `terraform apply`, Ansible).
- **Monitoring:** Prometheus mit GCE-Service-Discovery, Grafana mit provisionierten Dashboards, Loki mit Promtail als Log-Shipper, Blackbox Exporter für den DB-Integritätscheck.

2.3 Begründung der Technologieauswahl

Die Google Cloud Platform war durch die Projektvorgabe gesetzt. Die Auswahl der konkreten GCP-Dienste folgt jedoch der Architektur: Compute Engine bildet die IaaS-Schicht für App- und Monitoring-VMs, Firestore übernimmt als verwalteter PaaS-Dienst die persistente Datenhaltung, Artifact Registry speichert die Container-Images, Cloud Storage hält Konfigurations- und Backup-Artefakte, und Cloud Scheduler stößt die geplanten Firestore-Exporte an. Dadurch werden alle drei Cloud-Service-Modelle sichtbar genutzt, ohne zusätzliche Eigenbetriebs-Komplexität einzuführen.

2.3.1 Backend: FastAPI vs. Flask und Django

FastAPI wurde gegenüber den naheliegenden Python-Alternativen Flask und Django bevorzugt. Flask ist zwar ähnlich leichtgewichtig, liefert aber weder Request-Validierung noch OpenAPI-Generierung out-of-the-box; beides müsste über Zusatzbibliotheken (z.B. `flask-pydantic`, `flaskgger`) nachgerüstet werden. Django bringt ORM, Admin und Templating mit, die im Projekt nicht gebraucht werden, da Firestore als Datenhaltung fungiert und das Frontend als SPA getrennt läuft. FastAPI ist dagegen explizit auf typsichere JSON-APIs ausgelegt: Pydantic-Modelle dienen zugleich als Validierungsschicht und als Quelle der automatisch generierten OpenAPI-Spezifikation, und der ASGI-Unterbau (Starlette + Uvicorn) liefert asynchrone I/O, die bei Gemini-Aufrufen mit typisch mehreren Sekunden Antwortzeit relevant ist [1]. Unabhängige Benchmarks (TechEmpower Round 22) zeigen FastAPI/Uvicorn im JSON-Durchsatz mehrfach vor Flask [2]. Für ein Projekt mit Chat-, Auth-, Health- und Metrik-Endpunkten ergibt das bei gleicher Codemenge ein strengeres Vertragsmodell zwischen Frontend und Backend.

2.3.2 Frontend: React + Vite vs. Create-React-App und Next.js

Für das Frontend wurde React mit Vite statt des historisch üblichen Create-React-App (CRA) oder eines Full-Stack-Frameworks wie Next.js gewählt. CRA wurde von seinen Maintainern 2023 abgekündigt und erhält keine aktiven Updates mehr; Vite gilt seitdem als de-facto Nachfolger für SPA-Builds und bietet dank ESM-basiertem Dev-Server Start- und HMR-Zeiten im Millisekundenbereich [3]. Next.js wäre mit SSR und API-Routen überdimensioniert: Das Backend existiert bereits als FastAPI-Dienst, und eine server-generierte Seite bringt für den eingeloggten Chat-Flow keinen Vorteil. Die statische Auslieferung des Vite-Builds über Nginx passt hingegen direkt in das bestehende Container-Modell.

2.3.3 Datenhaltung: Firestore vs. Cloud SQL und selbstbetriebene DB

Firestore wurde gegenüber den Alternativen Cloud SQL (PostgreSQL) und einer selbst auf einer VM betriebenen Datenbank bevorzugt. Die drei Hauptargumente sind Datenmodell, Betriebsaufwand und Kosten:

- **Datenmodell.** Nutzerprofile, verschachtelte Konversationen mit Nachrichten-Arrays und Reisepläne sind dokumentenförmig; sie in ein relationales Schema zu zwingen, würde Joins erzeugen, die im Zugriffsmuster gar nicht gebraucht werden. Die Subcollection-Struktur `users/{uid}/conversations/{id}` entspricht direkt dem UI-Zugriffspfad.
- **Transaktionen.** Das tägliche Gemini-Rate-Limit erfordert einen atomaren Read-Modify-Write auf `rate_limits/{uid}`. Firestore liefert dafür ACID-Transaktionen auf Dokumentenebene [4]; ein häufiges Gegenargument gegen NoSQL-Datenbanken greift an dieser Stelle also nicht.
- **Betrieb und Kosten.** Eine selbstbetriebene PostgreSQL-Instanz auf einer VM würde Backup, Patching und Monitoring zusätzlich zur eigentlichen Anwendung erfordern. Cloud SQL nimmt diese Aufgaben ab, verursacht aber bereits in der kleinsten Zone unabhängig von der tatsächlichen Nutzung laufende Kosten [5], während Firestore einen Free-Tier-Sockel (1 GiB Speicher, 50k Reads, 20k Writes pro Tag) bietet, der den Projektbetrieb abdeckt [6].

2.3.4 KI-Modell: Gemini 3.1 Flash-Lite vs. 2.5 Flash

Aufgrund der Nutzungslimits des Gratis Tiers der Gemini API [7] kamen nur zwei Modelle in Frage: Gemini 2.5 Flash und Gemini 3.1 Flash-Lite (zur Zeit dieses Projekts noch Preview). Gemini 3.1 Flash-Lite (momentan noch Preview) ist laut Google auf niedrige Latenz und hohen Durchsatz bei gleichzeitig günstigerem Preismodell ausgelegt und übertrifft in internen Benchmarks den Vorgänger Gemini 2.5 Flash bei Geschwindigkeit und Antwortqualität [8].

2.3.5 Container-Orchestrierung: Docker Compose vs. Kubernetes (GKE)

Docker Compose wurde gegenüber Kubernetes bewusst gewählt. GKE (Autopilot wie Standard) erhebt eine Cluster-Management-Gebühr von derzeit \$0,10 pro Stunde und Cluster [9], was rund \$73 pro Monat *vor* den eigentlichen Worker-Kosten bedeutet. Das Projekt wird über die monatlichen \$50 Education Credits finanziert, womit die reine GKE-Grundgebühr das Budget bereits überschreiten würde, bevor überhaupt eine Worker-VM läuft. Zusätzlich wäre die operative Komplexität (Manifeste, Ingress-Controller, RBAC, Cluster-Upgrades) für drei Container pro VM und zwei Instanzen insgesamt deutlich überdimensioniert. Die eigentlichen Kubernetes-Funktionen, die im Projekt gebraucht werden — horizontales Replizieren und Selbstheilung — übernimmt die Managed Instance Group mit HTTP-Health-Check und Rolling-Replace [10]. Compose bleibt damit auf die VM-lokale Orchestrierung von Frontend-, Backend- und Promtail-Containern beschränkt, wo es eindeutig das einfachste Werkzeug ist.

2.3.6 IaC und Konfigurationsmanagement: Terraform und Ansible

Terraform und Ansible waren durch die Modulvorgabe gesetzt; die inhaltliche Entscheidung betraf daher weniger *ob*, sondern *wo* jedes der beiden Werkzeuge eingesetzt wird. Die Trennung folgt den jeweiligen Stärken: Terraform beschreibt mit dem `hashicorp/google`-Provider deklarativ den Zielzustand der Infrastruktur (Netzwerk, Load Balancer, Managed Instance Group, Firestore, IAM, Buckets, Scheduler) und verwaltet über seinen State Abhängigkeiten zwischen diesen Ressourcen [11]. Ansible ist dagegen prozedural-imperativ, arbeitet agentenlos über SSH [12] und eignet sich damit gut für die Konfiguration einer bestehenden, lange laufenden VM, ohne dort dauerhafte Agenten-Software zu installieren.

Im Projekt wird Ansible deshalb ausschließlich für die persistente Monitoring-VM eingesetzt, auf der Prometheus-, Grafana- und Loki-Konfiguration über eine Rolle verwaltet werden. Die App-VMs werden bewusst *nicht* mit Ansible provisioniert, sondern über Instance Template und Startup-Script bootstrapped, damit neu erzeugte MIG-Instanzen ohne nachträglichen SSH-

Eingriff einsatzfähig werden. GitHub Actions verbindet diese Schritte zu einer Pipeline aus Build, Image-Push, `terraform apply` und Ansible-Lauf für die Monitoring-VM.

2.3.7 Monitoring: Prometheus, Grafana und ergänzend Loki

Prometheus und Grafana waren als Werkzeuge vorgegeben. Loki wurde ergänzend aufgenommen, weil es aus demselben Grafana-Labs-Ökosystem stammt und sich nahtlos als Datenquelle in Grafana einbinden lässt — dadurch liegen Metriken und Logs in derselben Oberfläche, ohne zusätzlichen Tool-Wechsel.

In der MIG-Topologie ist Prometheus besonders passend, weil die offizielle `gce_sd_config`-Service-Discovery neu erzeugte Instanzen automatisch als Scrape-Targets erkennt und bei Replacement-Ereignissen entfernt [13]; statische Target-Listen wären bei Rolling-Replace fragil. Der Blackbox Exporter ergänzt das Setup um *aktive* Prüfungen des `GET /api/health/db`-Endpunkts, die unabhängig vom Nutzertraffic laufen und so auch bei geringer Last Fehlzustände sichtbar machen [14]. Promtail reicht als Log-Shipper, weil auf den App-VMs lediglich Container- und System-Logs abgeholt werden; eine OpenTelemetry-basierte Pipeline wird deshalb nicht als aktueller Stand, sondern als Ausblick behandelt.

3 Architektur und Umsetzung

3.1 Gesamtarchitektur

Abbildung 1 zeigt die finale Architektur nach der Migration auf einen Load-Balancer-basierten Betrieb mit $N > 1$ App-Instanzen. Der öffentliche Einstiegspunkt ist eine reservierte globale IP-Adresse, die von einem externen HTTP(S)-Load-Balancer gehalten wird. Dieser verteilt Traffic auf eine Managed Instance Group mit zwei App-VMs. Der Monitoring-Stack läuft getrennt auf einer persistenten VM mit eigener Persistent-Disk.

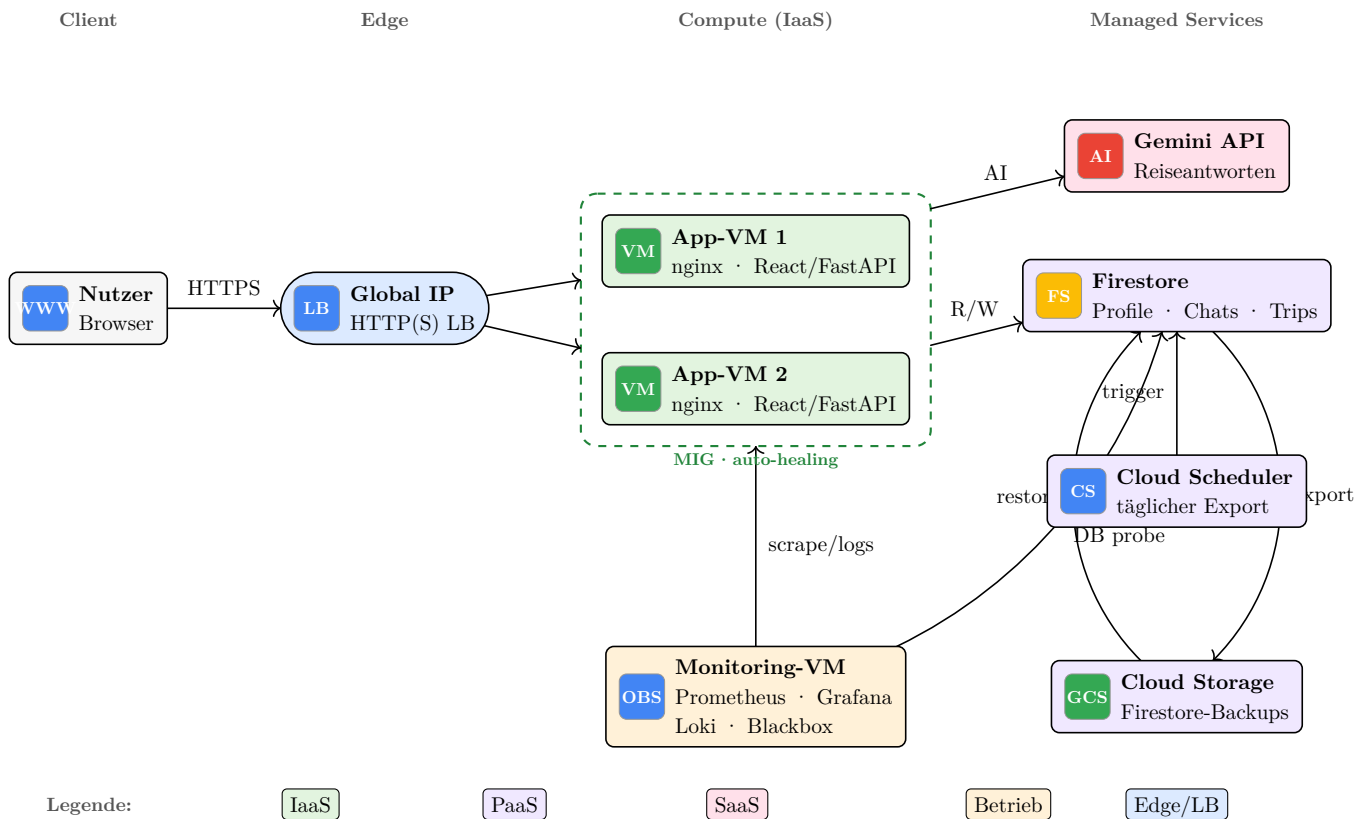


Abbildung 1: Gesamtarchitektur mit getrennten Pfaden: Der Nutzerverkehr läuft horizontal über Browser, globale IP und Load Balancer in die Managed Instance Group. Firestore und Gemini sind als direkte Laufzeitabhängigkeiten sichtbar, während Monitoring und Backup/Restore als eigene Betriebsflüsse räumlich getrennt sind.

3.2 Request-Pfad und Deployment-Flüsse

Die Architektur trennt bewusst den *Request-Pfad* vom *Deployment-Pfad*. Abbildung 2 macht diese Trennung explizit. Produktions-Traffic wird ausschließlich über den Load Balancer in die MIG geleitet, während das CI/CD-System über Terraform und die Instance-Template-Rotation neue Versionen ausrollt.

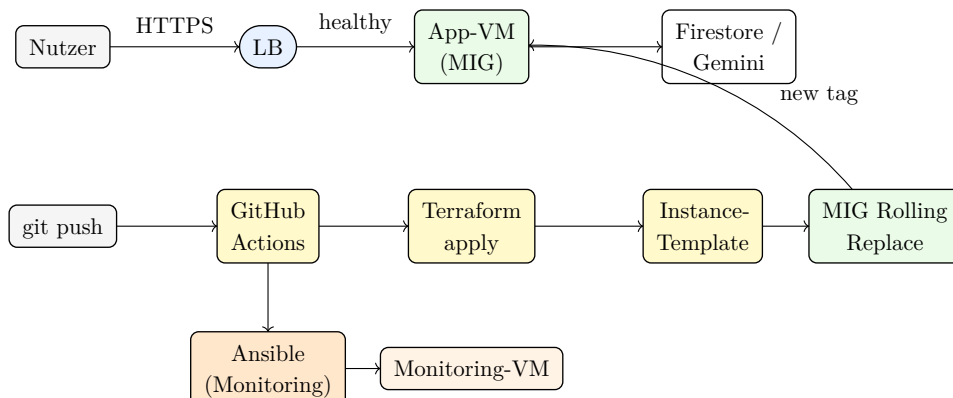


Abbildung 2: Getrennte Pfade für Laufzeit-Requests (oben) und Deployment (unten). Ansible wird nur für die persistente Monitoring-VM verwendet.

3.3 Entwicklung des Deployment-Modells

Die heutige Trennung *Terraform + Startup-Script für App*, *Ansible für Monitoring* ist das Ergebnis einer iterativen Designentwicklung. Zunächst war sowohl App- als auch Monitoring-Deployment über Ansible organisiert. Dieses Modell wurde aus drei Gründen abgelöst:

1. Mit $N > 1$ App-Instanzen würde ein SSH-basiertes Deployment in eine konkrete VM zwangsläufig Konfigurationsdrift erzeugen.
2. Eine im MIG neu gestartete Ersatz-VM muss ohne externes Eingreifen einsatzbereit werden – ein Nachzug über Ansible wäre ein Single Point of Operator Intervention.
3. Das Image-Tagging auf `latest` triggerte kein zuverlässiges Instance-Template-Update. Durch die Umstellung auf den Git-SHA als Image-Tag entsteht pro Commit eine eindeutige Template-Version, die den MIG-Rollout sicher auslöst.

Konkret wurden folgende Änderungen umgesetzt:

- Öffentlicher Einstieg auf reservierte globale Load-Balancer-IP umgestellt.
- App-MIG auf zwei Instanzen mit proaktivem Rolling-Replace erweitert.
- Startup-Script als autoritativer App-Bootstrap: holt Docker-Compose, `.env` und Promtail-Konfiguration aus einem GCS-Bucket, authentifiziert sich gegen Artifact Registry und startet den Compose-Stack.
- Legacy-Ansible-Rollen für die App entfernt, um nur noch *eine* Deployment-Geschichte im Repository zu halten.

3.4 Monitoring-Stack

Der Monitoring-Stack läuft auf einer eigenen VM. Das ist bewusst gewählt, damit die Observability nicht mit der überwachten Infrastruktur verschwindet:

- **Prometheus** entdeckt Scrape-Targets über GCE-Service-Discovery. Ein statischer Target-Eintrag wäre bei einer rollierenden MIG fragil.
- **Blackbox Exporter** prüft `GET /api/health/db` regelmäßig und exportiert `probe_success`. Der Endpoint wird also *aktiv* und *unabhängig* von App-Traffic beobachtet.
- **Grafana** lädt Dashboards automatisch aus einer provisionierten Verzeichnisstruktur. Ein initialer Fehler bei der Volume-Mount-Konfiguration (siehe Kapitel Abschnitt 5.3) wurde behoben, indem das Dashboard-Verzeichnis explizit in den Container gemountet wurde.
- **Loki** nimmt Logs von Promtail entgegen, welches auf jeder App-VM Container- und System-Logs einsammelt.
- **Persistente Metriken und Logs** liegen auf einer separaten Persistent Disk mit `prevent_destroy = true`, damit sie VM-Neustarts und Redeploys überleben.

4 Fokus-Feature: Disaster Recovery

4.1 Leitidee: zwei Recovery-Pfade

Das gewählte Fokus-Feature ist ein klar strukturiertes Disaster-Recovery-Konzept. Die zentrale Entscheidung: *Compute-Recovery* und *Datenrecovery* werden als zwei getrennte Recovery-Pfade behandelt, weil sie unterschiedliche Fehlerklassen adressieren (Abbildung 3).

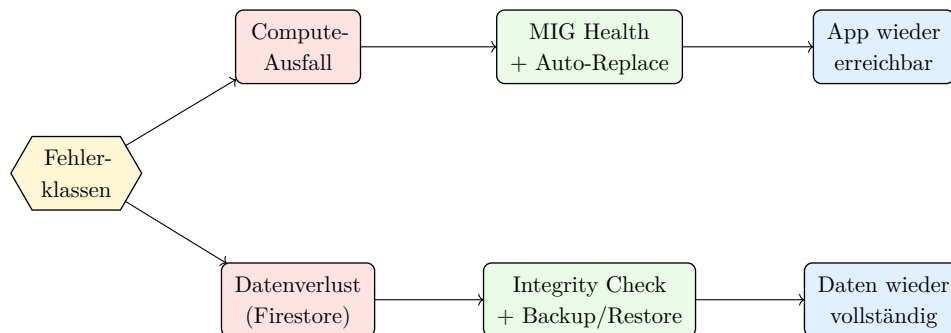


Abbildung 3: Zwei-Pfad-Recovery: Compute-Fehler werden durch die MIG behandelt, Datenfehler durch Integritätscheck, Backup und Restore.

Diese Trennung ergibt sich aus einer einfachen Beobachtung: Eine MIG heilt nur *Infrastruktur*, nicht *Daten*. Würde man ausschließlich auf MIG-Autohealing setzen, bliebe ein Szenario wie ein versehentlich gelöschttes Firestore-Dokument unbehandelt, obwohl die App-Instanzen technisch gesund sind.

4.2 Compute-Recovery über die MIG

Jede App-Instanz wird über einen HTTP-Health-Check gegen `/api/health` überwacht. Fällt eine Instanz aus, ersetzt die MIG sie automatisch anhand des aktuellen Instance-Templates. Das Startup-Script bootstrapt die Ersatz-Instanz ohne manuellen Eingriff. Die globale Load-Balancer-IP bleibt dabei unverändert, weil sie nicht an eine konkrete VM gebunden ist.

4.3 Datenrecovery

4.3.1 Deterministischer Integritätscheck

Ein bloßer Ping auf Firestore würde einen Datenverlust nicht erkennen, da die Datenbank technisch weiterhin antwortet. Deshalb wurde ein dedizierter Endpoint `GET /api/health/db` entwickelt, der konkret die Existenz *und* ein paar Pflichtfelder fester Referenzdokumente überprüft:

- `users/demo-user` (Felder: `display_name`, `email`)
- `users/demo-user/trips/demo-trip` (Felder: `destination`, `start_date`, `end_date`, `status`)
- `analytics/system` (Felder: `seed_version`, `last_seeded_at`)

Fehlt eines dieser Dokumente oder ein Pflichtfeld, liefert der Endpoint `HTTP 500` mit einer strukturierten Fehlerliste und loggt ein Ereignis `db_integrity_error`. Ein Blackbox Exporter fragt den Endpoint regelmäßig ab und exportiert `probe_success` als Prometheus-Metrik. Ein Grafana-Stat-Panel *Database Integrity* zeigt den Zustand kontinuierlich, unabhängig von App-Traffic.

4.3.2 Backups und Restore

Die aktuelle Backup-Strecke ist bewusst als *Daily Export* umgesetzt: Der automatisierte Schutzpunkt entsteht einmal pro Tag, ergänzt durch manuelle Vorab-Backups für geplante Recovery-Tests. Diese Lösung ist einfach, prüfbar und passt zum Projektumfang, hat aber einen groben RPO: Änderungen zwischen zwei Scheduler-Läufen sind nicht durch den letzten geplanten Export abgedeckt. Firestore-Exporte eignen sich laut Google für das Wiederherstellen nach versehentlicher Löschung und werden über Cloud Storage abgelegt; ein Export ist jedoch kein exakt zum Startzeitpunkt eingefrorener Snapshot [15].

Die Terraform-Konfiguration legt drei zusammenhängende Ressourcen an:

1. Ein GCS-Bucket `${project-id}-firestore-backups` mit einer 30-Tage- Lifecycle-Regel.
2. Ein dedizierter Service Account mit den Rollen `datastore.importExportAdmin` und `storage.admin` für genau diesen Bucket.
3. Ein Cloud-Scheduler-Job, der täglich um 03:00 Uhr Europe/Berlin die Firestore-Export-API mit OAuth-Token dieses Service Accounts aufruft und nach `gs://.../scheduled/` schreibt.

Zusätzlich existiert ein manuelles Backup-Skript `scripts/firestore-backup.sh` für Vorab-Backups vor gezielten Recovery-Tests. Der Restore erfolgt über `scripts/firestore-restore.sh`, das entweder das jüngste manuelle Backup oder einen explizit angegebenen Backup-Pfad importiert. Der Import ist asynchron und kann mehrere Minuten dauern.

4.3.3 Recovery-Ablauf

Der Recovery-Ablauf eskaliert bewusst schrittweise, um die Zwei-Pfad-Logik sichtbar zu machen:

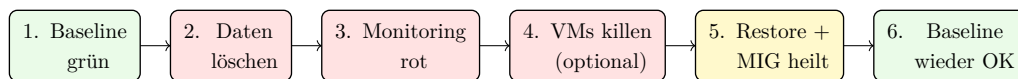


Abbildung 4: Staged Escalation des Recovery-Ablaufs: Daten werden vor Compute gekippt, damit der Unterschied zwischen App-Recovery und Daten-Recovery erkennbar wird.

4.4 Begründung einzelner Designentscheidungen

- **Warum sichtbare Datenlöschung statt eines versteckten Sentinel-Fehlers?** Eine Korruption, die nur im Backend sichtbar ist, ist schwer nachzuvollziehen. Durch gezieltes Löschen von Referenzdaten entsteht ein unmittelbarer UI-Effekt, der die Alarmierung nachvollziehbar macht.
- **Warum Cloud Scheduler statt VM-Cronjob?** Ein VM-lokaler Cronjob hätte dasselbe Ergebnis, ist aber architektonisch weniger passend: Cloud Scheduler + Managed-Export-API zeigt einen echten PaaS-Backup-Pfad, der VMs überlebt.
- **Warum ein eigener `demo-user`?** Der Integrity-Check darf sich nicht auf „irgendeinen Nutzer“, verlassen. Mit einem festen Referenzdatensatz wird der Recovery-Test reproduzierbar und die Alarmierung deterministisch.

4.5 Point-in-Time Recovery (Ausblick)

Eine vollständig zeitgenaue Wiederherstellung (Point-in-Time Recovery, PITR) wird von Firestore nativ unterstützt. Im Unterschied zum täglichen Export hält PITR ältere Dokumentversionen in einem Recovery-Fenster vor: ohne PITR ist nur ungefähr die letzte Stunde verfügbar, mit aktiviertem PITR bis zu sieben Tage; Lesezugriffe sind innerhalb der letzten Stunde mit Mikrosekundenpräzision und darüber hinaus innerhalb des PITR-Fensters minuten-genau möglich [16]. Damit ist PITR deutlich granularer als der hier umgesetzte Daily Export.

Im Rahmen dieses Projekts wurde PITR nicht aktiviert, weil es zusätzliche Kosten verursacht und für die nachweisbare Backup-/Restore-Strecke nicht notwendig war. Für einen produktionsnäheren Betrieb wäre ein hybrider Ansatz sinnvoll: Daily Exports bleiben als langlebige, bucketbasierte Sicherung und als Grundlage für projektübergreifende Restores erhalten, während PITR die Lücke zwischen zwei geplanten Exporten schließt und versehentliche Schreib- oder Löschfehler feingranularer rückgängig machen kann. Architektonisch ließe sich PITR ohne

Änderungen am App-Code nachziehen: Es müsste in der Firestore-Konfiguration aktiviert werden, anschließend könnten zeitpunktbezogene Reads, Exporte oder Datenbank-Klone für feinere Recovery-Szenarien genutzt werden.

5 Ergebnisse und Diskussion

5.1 Umgesetzter Systemumfang

Bereich	Umsetzung
Web-Anwendung	E-Mail/Passwort-Login mit JWT, KI-Dialog mit persistenten Konversationen und speicherbaren Reiseplänen, Rate-Limit pro Nutzer.
Cloud-Service-Modelle	SaaS: Gemini API · PaaS: Firestore, Artifact Registry, Cloud Scheduler, Cloud Storage · IaaS: Compute Engine (MIG + Monitoring-VM), Load Balancer, Persistent Disks.
Infrastruktur	Komplette Infrastruktur inklusive MIG, LB, Firestore, IAM, Cloud-Storage-Buckets und Cloud-Scheduler-Job.
Konfiguration	Auf die persistente Monitoring-VM fokussiert; App-VMs werden über Startup-Script bereitgestellt.
Disaster Recovery	Sichtbare Datenkorruption, MIG-Autohealing, Grafana-Alert und Restore-Script.
Monitoring	Prometheus (GCE-SD), Grafana, Loki, Blackbox Exporter auf separater VM mit persistenter Disk.
Entwicklung und Betrieb	Typed FastAPI-Endpunkte, pytest-Testsuite, CI über GitHub Actions.
Rollout-Modell	GitHub Actions: Build/Push → <code>terraform apply</code> → Ansible (Monitoring) → Instance-Template-Rotation; globale statische IP vor HTTP(S)-LB, $N > 1$ App-Instanzen mit Rolling-Replace.

Tabelle 3: Zusammenfassung der umgesetzten Systembereiche.

5.2 Architektonische Iterationen

Die Commit-Historie des Repositories dokumentiert eine gradlinige Architekturentwicklung:

1. *Single-VM + lokales Monitoring.*
2. *Monitoring auf eigene VM ausgelagert.* Begründung: bessere Fehlertoleranz, klarere Verantwortlichkeiten.
3. *Self-healing MIG mit statischer VM-IP.* Begründung: Resilienz ohne zusätzliche Load-Balancer-Komplexität.
4. *Statische IP an Load Balancer verschoben, MIG auf $N = 2$.* Begründung: konsistentes $N > 1$ -Modell und stabile öffentliche Adresse.
5. *App-Deployment aus Ansible gelöst, in Instance-Template + Startup-Script überführt.* Begründung: reproduzierbares, drift-freies Multi-Instance-Rollout.

Der Wechsel von einer direkt an die VM gebundenen statischen IP auf eine Load-Balancer-IP ist dabei bewusst als *Supersede*-Entscheidung markiert: das ältere Design war für einen einzelnen

VM-Autoheal-Zyklus tragfähig, wurde aber mit dem Übergang zu $N > 1$ architektonisch zur falschen Abstraktion.

5.3 Beobachtete Probleme und Lessons Learned

- **Startup-Script zu früh gestartet.** Beim ersten LB-Rollout lief die Startup-Sequenz einer Ersatz-VM an, bevor die aktualisierten Deployment-Dateien im GCS-Bucket lagen. Der Rollout erholte sich bei der nächsten Runde, dokumentiert aber einen Bedarf für Retry-Logik bei GCS-Downloads.
- **Backend-Healthcheck ohne `curl`.** Der Docker-Compose-Healthcheck nutzte `curl`, das im Backend-Image nicht enthalten war. Fix: `curl` im Dockerfile ergänzen. Lesson: Healthchecks verlangen dieselbe Toolkette wie die Anwendung.
- **Grafana-Dashboards nicht provisioniert.** Die Provisioning-Konfiguration zeigte auf `/etc/grafana/dashboards`, aber der Volume-Mount legte nur Teilpfade ab. Fix: das Dashboard-Verzeichnis explizit mounten.
- **cloudscheduler.googleapis.com nicht aktiviert.** Ein neu eingeführter Scheduler-Job schlug fehl, bis die API projektweit aktiviert und dem CI/CD-Service-Account die Rolle `serviceusage.serviceUsageAdmin` zugewiesen wurde. Lesson: Disaster Recovery ist nicht nur Architektur, sondern auch *Service-Account-Hygiene*.

5.4 Grenzen der aktuellen Lösung

- Das Restore ist manuell angestoßen. In einem Produktionsbetrieb wäre ein definierter RTO/RPO-Zielwert mit automatisierter Restore-Entscheidung wünschenswert.
- Die geplanten Firestore-Backups laufen nur täglich; PITR ist konzeptionell vorgesehen, aber aus Kostengründen nicht aktiviert. Dadurch bleibt der Recovery-Punkt zwischen zwei Daily Exports gröber als in einem hybriden Export-plus-PITR-Setup.
- Das Logging ist aktuell klassisch über Promtail nach Loki umgesetzt, nicht über OpenTelemetry. Ein OTel-Collector mit OTLP-Empfang wäre eine sinnvolle Erweiterung, um Logs, Metriken und Traces stärker zu standardisieren und vendor-neutral weiterzugeben.
- Die MIG betreibt zwei Instanzen in *einer* Region. Ein echtes Multi-Region-Setup würde einen deutlich größeren Aufwand bedeuten, wurde im Projektumfang aber nicht umgesetzt.

6 Fazit

Das Projekt realisiert eine cloud-native Reiseplanungsanwendung mit klar getrennten Verantwortlichkeiten: Alle drei Cloud-Ebenen sind produktiv im Einsatz, die Infrastruktur wird vollständig über Terraform bereitgestellt, Ansible übernimmt die Konfiguration des persistenten Monitoring-Hosts, und das Monitoring-Konzept zeigt sowohl Infrastruktur- als auch Datengesundheit an.

Das Fokus-Feature *Disaster Recovery* unterscheidet klar zwischen Compute-Recovery und Datenrecovery und macht diese Trennung im Recovery-Ablauf sichtbar. Die eingeführte Unterscheidung zwischen Integritätscheck und reinem Connectivity-Ping sowie das zweistufige Backup-Konzept (geplant + manuell) machen den Recovery-Pfad nachvollziehbar. Der Load Balancer und die CI/CD-Pipeline ergänzen diese Architektur um einen stabilen öffentlichen Einstiegspunkt und reproduzierbare Rollouts.

Für zukünftige Iterationen bieten sich vor allem drei Richtungen an: *automatisiertes Restore* (RTO/RPO-getrieben), ein hybrider *Daily-Export-plus-PITR*-Ansatz für Firestore, eine *OpenTelemetry*-basierte Observability-Pipeline und ein *Multi-Region-Failover* für den Load Balancer. Die aktuelle Trennung zwischen Terraform-gesteuertem App-Rollout und Ansible-gesteuertem Monitoring-Host bleibt dabei tragfähig und ermöglicht diese Erweiterungen, ohne die bestehende Deployment-Geschichte zu brechen.

Referenzen

- [1] Sebastian Ramirez, „FastAPI — Features: On par with NodeJS and Go, automatic OpenAPI, Pydantic-based validation“. Zugegriffen: 20. April 2026. [Online]. Verfügbar unter: <https://fastapi.tiangolo.com/features/>
- [2] TechEmpower, „Web Framework Benchmarks — Round 22“. Zugegriffen: 20. April 2026. [Online]. Verfügbar unter: <https://www.techempower.com/benchmarks/>
- [3] Vite, „Why Vite — Problems and how Vite addresses them“. Zugegriffen: 20. April 2026. [Online]. Verfügbar unter: <https://vite.dev/guide/why.html>
- [4] Google Cloud, „Firestore: Transactions and Batched Writes“. Zugegriffen: 20. April 2026. [Online]. Verfügbar unter: <https://cloud.google.com/firestore/docs/manage-data/transactions>
- [5] Google Cloud, „Cloud SQL Pricing“. Zugegriffen: 20. April 2026. [Online]. Verfügbar unter: <https://cloud.google.com/sql/pricing>
- [6] Google Cloud, „Firestore Pricing (Free Tier and Spark Plan)“. Zugegriffen: 20. April 2026. [Online]. Verfügbar unter: <https://cloud.google.com/firestore/pricing>
- [7] Google, „Google AI Studio — API Rate Limits (Free Tier)“. Zugegriffen: 20. April 2026. [Online]. Verfügbar unter: <https://aistudio.google.com/app/rate-limit>
- [8] Google, „Gemini 3.1 Flash-Lite: Built for intelligence at scale“. Zugegriffen: 20. April 2026. [Online]. Verfügbar unter: <https://blog.google/innovation-and-ai/models-and-research/gemini-models/gemini-3-1-flash-lite/>
- [9] Google Cloud, „GKE Pricing — Cluster Management Fee“. Zugegriffen: 20. April 2026. [Online]. Verfügbar unter: <https://cloud.google.com/kubernetes-engine/pricing>
- [10] Google Cloud, „Compute Engine: Managed Instance Groups“. Zugegriffen: 20. April 2026. [Online]. Verfügbar unter: <https://cloud.google.com/compute/docs/instance-groups>
- [11] HashiCorp, „Terraform Google Provider Documentation“. Zugegriffen: 20. April 2026. [Online]. Verfügbar unter: <https://registry.terraform.io/providers/hashicorp/google/latest/docs>
- [12] Red Hat, „How Ansible Works — Agentless architecture over SSH“. Zugegriffen: 20. April 2026. [Online]. Verfügbar unter: <https://www.ansible.com/how-ansible-works/>
- [13] Prometheus Authors, „Prometheus Configuration: gce_sd_config“. Zugegriffen: 20. April 2026. [Online]. Verfügbar unter: https://prometheus.io/docs/prometheus/latest/configuration/configuration/#gce_sd_config

- [14] Prometheus Authors, „Blackbox Exporter“. Zugegriffen: 20. April 2026. [Online]. Verfügbar unter: https://github.com/prometheus/blackbox_exporter
- [15] Google Cloud, „Firestore: Export and Import Data“. Zugegriffen: 20. April 2026. [Online]. Verfügbar unter: <https://cloud.google.com/firestore/docs/manage-data/export-import>
- [16] Google Cloud, „Firestore: Point-in-Time Recovery“. Zugegriffen: 20. April 2026. [Online]. Verfügbar unter: <https://cloud.google.com/firestore/docs/use-pitr>
- [17] Google Cloud, „External HTTP(S) Load Balancing Overview“. Zugegriffen: 20. April 2026. [Online]. Verfügbar unter: <https://cloud.google.com/load-balancing/docs/https>
- [18] Google Cloud, „Cloud Scheduler Documentation“. Zugegriffen: 20. April 2026. [Online]. Verfügbar unter: <https://cloud.google.com/scheduler/docs>
- [19] Red Hat and Ansible Community, „Ansible Documentation“. Zugegriffen: 20. April 2026. [Online]. Verfügbar unter: <https://docs.ansible.com/>
- [20] Prometheus Authors, „Prometheus Documentation“. Zugegriffen: 20. April 2026. [Online]. Verfügbar unter: <https://prometheus.io/docs/introduction/overview/>
- [21] Grafana Labs, „Grafana and Loki Documentation“. Zugegriffen: 20. April 2026. [Online]. Verfügbar unter: <https://grafana.com/docs/>
- [22] Sebastian Ramirez, „FastAPI Documentation“. Zugegriffen: 20. April 2026. [Online]. Verfügbar unter: <https://fastapi.tiangolo.com/>
- [23] Grafana Labs, „Loki: Like Prometheus, but for logs“. Zugegriffen: 20. April 2026. [Online]. Verfügbar unter: <https://grafana.com/oss/loki/>
- [24] HashiCorp, „Terraform vs. Alternatives — Cloud-agnostic infrastructure as code“. Zugegriffen: 20. April 2026. [Online]. Verfügbar unter: <https://developer.hashicorp.com/terraform/intro/vs>